

A Two-Model, Condition-Aware Machine-Learning Pipeline for Engineering Thermostable PET-Degrading Enzymes

PET-Lab Project Report · Stability-Prediction Track · June 2026

Abstract. Plastics such as PET degrade fastest near 70 °C, yet most plastic-degrading enzymes denature well below that temperature. We present an integrated machine-learning pipeline that pairs **two complementary models**: (1) a **whole-protein screening model** that predicts an enzyme's melting temperature (T_m) directly from sequence, used to *select* the most heat-tolerant starting enzymes; and (2) a **mutation model** that predicts the stability change ($\Delta\Delta G$) of a point mutation, used to *optimize* the chosen enzyme. Both are built on ESM-2 protein language-model embeddings and a zoo of gradient-boosted / ensemble regressors, trained on a 1,001,888-row, leakage-audited stability dataset assembled from seven public sources. We evaluate each model on an **independent, homology-clean benchmark**: S669 for the mutation model (ensemble Pearson 0.39, 95% CI 0.32–0.46) and a newly-constructed BRENDA temperature-stability test set for the screening model (AUC 0.73, 95% CI 0.71–0.76). The aim is to **cut expensive wet-lab effort** by filtering and ranking candidates *in silico*; because only a short shortlist reaches the bench, we prioritize **precision** and operate at high-precision thresholds. The contribution is one of **integration, application, and data rigor** rather than a new algorithm.

Contents

1. Introduction & motivation
2. The two-model approach
3. Pipeline stages (data prep · feature engineering · training · validation)
4. Datasets — training and test
5. Results
6. Limitations
7. Conclusion — why the integration is novel

1 · Introduction & Motivation

Enzymatic recycling of PET (polyethylene terephthalate) is most efficient at elevated temperatures (~70 °C), where the polymer becomes flexible and accessible. The bottleneck is biological: naturally-occurring PET

hydrolases (e.g. IsPETase) lose their fold and activity far below this temperature. Engineering enzymes that remain *stable and active under harsh temperature and pH* is therefore the central problem.

Two distinct questions arise. First, **which enzyme should we start from?** — a ranking problem over thousands of candidate sequences, the vast majority of which have *no measured stability*. Second, **how do we improve the chosen enzyme?** — a design problem over the space of point mutations. These are different questions requiring different models. Our pipeline answers both and chains them together, and — critically — validates each half on data the models never saw.

Goal: reduce expensive wet-lab effort. Characterizing an enzyme's stability experimentally costs days of work and significant money per candidate, so testing every option is infeasible. The pipeline's job is to **filter and rank** the candidate space in silico and forward only the most promising designs to the bench. For this purpose **precision matters more than recall**: it is far better to hand the lab a *short, trustworthy* shortlist (few false positives) than a long list padded with candidates that fail — missing some good enzymes (lower recall) is an acceptable price for not wasting bench time on bad ones. We therefore operate the models at **high-precision thresholds**, and report the full precision–recall trade-off so the operating point can be chosen deliberately.

2 • The Two-Model Approach

STAGE 1 • SCREEN

Whole-protein T_m model — “the scout”

Input: one sequence. Output: predicted melting temperature (absolute, °C).

Answers “*which enzyme is worth our time?*”
Predicts heat tolerance for enzymes that were never measured in a lab, so a large candidate pool can be ranked in silico and narrowed to the sturdiest scaffolds.

STAGE 2 • OPTIMIZE

Mutation $\Delta\Delta G$ model — “the engineer”

Input: one sequence + one mutation. Output: stability change (relative, $\Delta\Delta G$).

Answers “*how do we upgrade the one we picked?*” Scores candidate point mutations so the stabilizing ones can be combined into an improved design.

The two are complementary because one is **absolute** and the other is **relative**. Chained: the screening model gives the starting enzyme's T_m ; the mutation model gives the boost from chosen mutations; together they estimate the engineered enzyme's final stability ($T_m(\text{design}) \approx T_m(\text{WT}) + \Delta T_m(\text{mutations})$) — an absolute number you cannot obtain from the mutation model alone. Only the top designs proceed to wet-lab confirmation.

① Candidates

Many enzyme sequences, most with no measured stability.

② Screen · Stage 1

T_m model ranks them
→ pick the sturdiest scaffold.

③ Optimize · Stage 2

$\Delta\Delta G$ model scores mutations → keep the stabilizing ones.

④ Combine

$T_m(WT) + \Delta T_m \rightarrow$
estimated stability of the engineered design.

⑤ Confirm

Lab-test only the top-ranked designs.

3 · Pipeline Stages

Both models share the same four-stage skeleton — **data preparation** → **feature engineering** → **training** → **validation** — but differ in input granularity (whole protein vs. mutation pair).

3.1 Data preparation

- **Shared staging.** Seven public sources are merged into one richly-annotated table (`build_staging_table.py`) with full provenance per row (sequence, mutation, T, pH, measurement type, source, PMID). A **strict leakage audit** removes exact duplicates and any row overlapping the S669 test set — including **sequence-homology overlap** detected with `mmseqs2` ($\geq 30\%$ identity, $\geq 50\%$ coverage), which exact-key matching misses.
- **Stage 2 (mutation):** keep only $\Delta\Delta G$ rows that have *both* a wild-type and a mutant sequence (508,693 rows).
- **Stage 1 (screening):** keep T_m rows (single sequence → temperature); 23,349 unique sequences from Meltome + FireProtDB.

3.2 Feature engineering

Both use **ESM-2** (`esm2_t30_150M`, 640-dim, mean-pooled per residue) — a protein language model that encodes evolutionary and structural context from sequence alone (no 3-D structure required).

STAGE 2 — MUTATION (276-DIM)

- Embeddings of [wt, mut, mut-wt] → PCA to 256 dims (keeps 500K+ rows tractable).
- **Engineered block (not PCA'd)**: ESM-2 Δ LL (log-likelihood ratio of the substitution, the mutation-local signal); 12-d biochemical Δ (charge, volume, hydrophobicity, H-bonds, polarity, aromaticity, Pro/Gly); assay conditions (T, pH).

STAGE 1 — SCREENING (641-DIM)

- Whole-protein ESM-2 embedding (640-d) + log(sequence length).
- No PCA (the T_m set is small enough to use full embeddings).

3.3 Training

Both stages train a **model zoo** of gradient-boosted and ensemble regressors (XGBoost, LightGBM, CatBoost, Random Forest, Extra Trees, MLP) and average them into an ensemble. Embeddings are computed once on GPU (cached) on the Nautilus / NRP Kubernetes cluster; the regressors train on CPU. Splits are **protein-grouped** so no wild-type sequence appears on both sides of the train/validation boundary.

3.4 Validation

Each model is validated twice: an internal **grouped held-out set**, and an **external, homology-audited benchmark the model never saw** — **S669** for the mutation model and the **BRENDA temperature-stability set** for the screening model (Section 4.2).

4 • Datasets

4.1 Training data

The training table contains **1,001,888 rows** assembled from seven public stability resources and de-duplicated/leakage-audited. Because the sources measure different physical quantities, each row carries a **measurement type**, and each model trains only on the relevant type.

Rows by source dataset

Source	Rows
Domainome (aPCA abundance)	457,943
Tsuboyama 2023 (mega-scale)	357,155
Tsuboyama 2023 (doubles)	138,275
Meltome Atlas (T_m)	27,884
ThermoMutDB	11,520
FireProtDB	6,549
ProDDG / S2648	2,562

Rows by measurement type

Type	Rows	Used by
$\Delta\Delta G$ (mutation)	508,693	Stage 2
abundance (proxy)	457,943	—
T_m (whole protein)	29,654	Stage 1
ΔT_m	5,598	—

Key training stat: the leakage audit removed ~132,000 rows overlapping S669, of which **131,479 were homology-only** — invisible to exact UniProt/PDB keys and catchable only by mmseqs2. This is the single most important step for an honest generalization estimate.

4.2 Test data (independent benchmarks)

S669 — MUTATION MODEL

669 single-point mutations with experimental $\Delta\Delta G$; the field-standard benchmark for stability-change prediction. Audited (exact + homology) against the training set so no train mutation or close homolog leaks in.

BRENDA TEMPERATURE-STABILITY SET — SCREENING MODEL (NEW IN THIS WORK)

We constructed a fresh whole-protein benchmark from the **BRENDA 2026.1** bulk release (CC BY 4.0; 8,129 EC entries, 111,901 protein records). Three steps:

1. **Extract** (`convert_brenda.py`): keep proteins with a pH- and/or temperature-stability annotation. BRENDA stores no sequences inline — only UniProt accessions — so sequences are resolved from UniProt. → **3,060 enzymes** (2,886 temperature, 1,176 pH, 1,002 both; 2,917 accessions, 1,229 organisms, 868 EC numbers; **46% hydrolases**, the family of PET-degrading enzymes).
2. **Parse** (`parse_brenda_temp_stability.py`): convert the free-text temperature-stability annotations (e.g. “70 [10 min, stable]”, “ T_m 76.8 °C”) into a numeric ceiling and a label (**positive** ≥ 60 °C / **negative** ≤ 45 °C / ambiguous). A heuristic separates true T_m , “stable-up-to”, and inactivation evidence, and discards cold-storage, sub-ambient, immobilized, and implausible values. Parsed 84.8% (2,446).
3. **Leakage audit** (`brenda_leakage_audit.py`): remove any protein overlapping the training set by accession, exact sequence, or mmseqs2 homology ($\geq 30\%$ id, $\geq 50\%$ cov). Removed **412 (16.8%)** — of which **406 were homology-only**, again invisible to exact keys.

Final clean BRENDA test set: 2,034 enzymes (979 positive / 584 negative / 471 ambiguous) → **1,563 confident binary cases**, provably free of exact and homology overlap with training.

5 • Results

Results follow the pipeline order: the screening model (Stage 1, run first) then the mutation model (Stage 2).

5.1 Stage 1 — Screening model on BRENDA

Internal held-out validation: T_m predicted within **RMSE $\approx$ 6.5 °C** (val Pearson $\approx$ 0.80 across models) — the model learned “sequence → heat tolerance”.

Independent BRENDA test (1,563 unseen enzymes): AUC = 0.732 (95% CI 0.708–0.755) | regression Pearson 0.54 (95% CI 0.50–0.57; 0.60 on the true- T_m subset). The tight interval (n=1,563) places the model clearly above chance on proteins — and protein families — it never trained on.

BRENDA stable/unstable classification — threshold sweep (979 positive / 584 negative; AUC = 0.732)

Threshold	Precision	Recall	F1	Accuracy	Note
46 °C	0.65	0.98	0.78	0.66	best F1 / high recall
50 °C	0.71	0.77	0.74	0.66	balanced (P$\approx$R)
52 °C	0.77	0.63	0.69	0.65	
60 °C	0.93	0.39	0.55	0.60	trustworthy shortlist
66 °C	0.98	0.32	0.49	0.58	cleanest shortlist

For the screening use case (shortlisting a few candidates for the lab), the **high-threshold / high-precision** end is preferred: at 66 °C the model is right ~98% of the time when it calls an enzyme heat-stable, accepting lower recall by design.

Per-model classification at the default 60 °C threshold (precision = of enzymes flagged stable, fraction truly stable)

Model	Precision	Recall	F1
extra_trees	0.963	0.369	0.533
random_forest	0.948	0.355	0.517
lightgbm	0.933	0.395	0.555
catboost	0.927	0.400	0.559
xgboost	0.917	0.394	0.551
Ensemble	0.945	0.388	0.550

Precision is uniformly high (0.92–0.96) across every model — the screener's shortlist is ~93–96% trustworthy, which is exactly the property the wet-lab use case needs.

5.2 Stage 2 — Mutation model on S669

Bootstrap 95% CIs (2,000 resamples of the saved predictions): ensemble **Pearson 0.39 (0.32–0.46)**, stabilizing-**AUC 0.67 (0.62–0.72)**. Wider than BRENDA because S669 is smaller (n=669), but the lower bounds stay clearly above zero/chance.

S669 regression — predicted vs. experimental $\Delta\Delta G$ (669 mutations)

Model	Pearson	Spearman	RMSE (kcal/mol)
extra_trees (best single)	0.436	0.461	1.481
xgboost	0.382	0.413	1.514
random_forest	0.369	0.396	1.523
lightgbm	0.330	0.364	1.549
catboost	0.324	0.362	1.557
mlp	0.298	0.338	1.612
Ensemble	0.390	0.414	1.509

S669 as a stabilizing-mutation classifier (positive = $\Delta\Delta G < 0$; 168/501 split; AUC = 0.669) — threshold sweep

Threshold ($\Delta\Delta G$)	Precision	Recall	F1	Accuracy	Note
0.0 (strict)	0.81	0.10	0.18	0.77	clean picks
+0.25	0.51	0.26	0.35	0.75	
+0.5	0.38	0.46	0.42	0.67	balanced (P$\approx$R)
+1.0	0.31	0.80	0.45	0.51	best F1 / high recall

Per-model classification at the default $\Delta\Delta G < 0$ threshold (precision = of mutations flagged stabilizing, fraction truly stabilizing)

Model	Precision	Recall
random_forest	0.900	0.107
extra_trees	0.786	0.065
xgboost	0.735	0.149
lightgbm	0.625	0.119
catboost	0.429	0.107
mlp	0.415	0.202
Ensemble	0.810	0.101

Precision is more variable here (0.42–0.90); the ensemble reaches **0.81**. As with the screener, high precision is paired with low recall — the model is cautious, so its few calls are usually right but it misses most stabilizers (raise the threshold for recall, per the sweep above).

5.3 Precision–recall trade-off (both models)

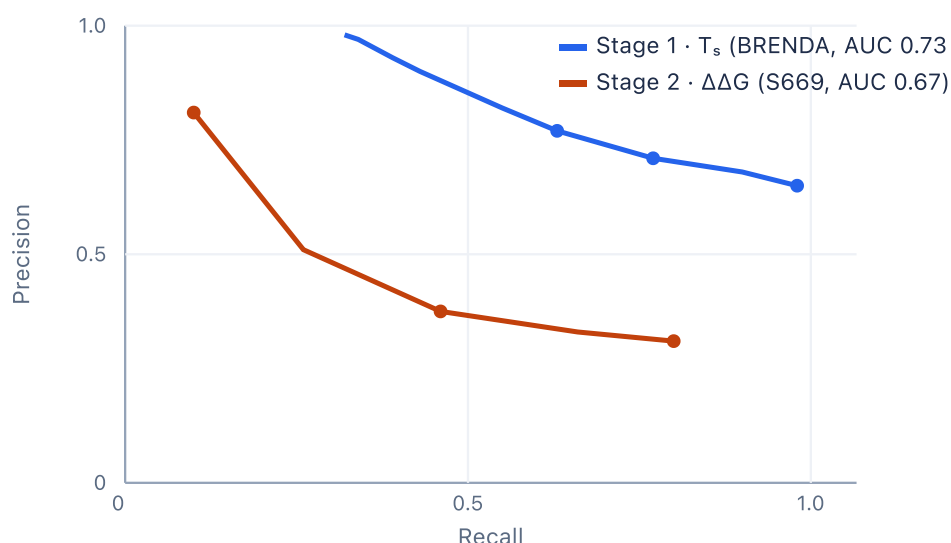


Figure 1. Precision vs. recall as the decision threshold is swept. The screening model (blue) bows toward the top-right — it sustains high precision across a useful recall range. The mutation model (orange) collapses toward the baseline — precision is high only at very low recall, the expected behavior for an embedding-based $\Delta\Delta G$ baseline on the hard S669 set.

6 • Limitations

- **Modest absolute accuracy.** S669 Pearson (0.44 best) is below specialized structure-aware $\Delta\Delta G$ methods (~ 0.5 – 0.6); the BRENDA AUC (0.73) is a useful-but-not-strong screener. Both are **tree-on-embedding baselines**, not fine-tuned language models.
- **Class imbalance.** Both benchmarks are imbalanced, so F1 must be read alongside AUC; “best-F1” thresholds lean toward recall.
- **Label noise in BRENDA.** The numeric ceiling is parsed from free text and mixes true T_m with “stable-up-to” values; a `basis` flag allows filtering to the cleaner subset.
- **pH not yet modeled.** The screening model currently predicts temperature stability; pH-stability annotations are extracted but not yet numeric.

7 • Conclusion — Why the Integration Is Novel

The individual components here are established: language-model embeddings, $\Delta\Delta G$ prediction, and sequence-to- T_m prediction all exist in the literature. **The novelty is in the integration, the application, and the data rigor** — i.e. it is a systems contribution:

- **Two complementary models in one loop.** An *absolute* screening model and a *relative* mutation model are combined into a single **screen-then-optimize** pipeline that produces an estimated absolute stability for a *designed* enzyme — neither model achieves this alone.
- **Application focus.** The pipeline targets **thermostable PET-degrading enzymes** specifically, where the training data is enriched (46% hydrolases) and the goal (function at ~70 °C) is concrete.
- **A new, reusable benchmark.** We repurpose BRENDA's free-text stability annotations into a numeric, labeled, **homology-clean** independent test set — a use of BRENDA not previously available for whole-protein stability evaluation.
- **Leakage discipline.** Both benchmarks are audited for *sequence-homology* leakage with mmseqs2, not just exact-key overlap. In this project that step alone removed 130k+ training rows and ~17% of the BRENDA test set — overlap that would otherwise silently inflate every reported score.

In short: rather than a new algorithm, the project delivers a **carefully-integrated, condition-aware, leakage-audited pipeline** that turns two known prediction tasks into a usable enzyme-engineering workflow, with honest, independent validation on both halves.

Reproducibility. Training data: `stability_dataset_multitask.csv` ; staging: `build_staging_table.py` · `run_mmseqs_audit.py` . BRENDA benchmark: `convert_brenda.py` · `parse_brenda_temp_stability.py` · `brenda_leakage_audit.py` . Models: Stage 2 `build_features.py` + `train_gpu/cpu_models.py` (eval `ensemble_eval.py` , `s669_sweep.py`); Stage 1 `build_features_tm.py` + `train_tm_models.py` (eval `predict_and_sweep.py`). Embeddings: ESM-2 `esm2_t30_150M` . Compute: Nautilus/NRP Kubernetes (GPU embed, CPU train).